Here are the complete details (Statement, Hardware Details, Case Study Working, Interfacing Description, and Pseudo Code) for all 25 problem statements based on the reference document.

1. Automatic Corridor Light

- **1) Statement:** A college corridor has an LED indicator that needs to be controlled using Arduino UNO. Write a program to turn the LED ON for 2 seconds and OFF for 1 second continuously.
- **2) Hardware Details:** Arduino UNO, 1x LED, 1x 220Ω resistor, breadboard, connecting wires.
- **3) Case Study Working:** The microcontroller toggles the digital output pin connected to the LED between HIGH and LOW states using delay functions, creating a repeated 2-second ON and 1-second OFF cycle.
- **4) Schematic / Interfacing Diagram:** Arduino Digital Pin 8 \rightarrow 220Ω Resistor \rightarrow LED Anode. LED Cathode \rightarrow Arduino GND.
- **5) Pseudo Code:**

```
void setup() { pinMode(8, OUTPUT); }
void loop() {
  digitalWrite(8, HIGH); delay(2000);
  digitalWrite(8, LOW); delay(1000);
}
```

2. Emergency Warning Indicator

- **1) Statement:** Design an emergency warning system using an LED. The LED should blink continuously at a rate of one blink every 500 ms. Modify the program so that the blinking pattern represents an emergency warning signal.
- **2) Hardware Details:** Arduino UNO, 1x LED, 1x 220Ω resistor, breadboard, wires.
- **3) Case Study Working:** The LED flashes rapidly with equal 250 ms ON and 250 ms OFF intervals (total 500 ms period) to signify an active emergency alert status.
- **4) Schematic / Interfacing Diagram:** Arduino Digital Pin 9 \rightarrow 220Ω Resistor \rightarrow LED Anode. LED Cathode \rightarrow GND.
- **5) Pseudo Code:**

```
void setup() { pinMode(9, OUTPUT); }
void loop() {
  digitalWrite(9, HIGH); delay(250);
  digitalWrite(9, LOW); delay(250);
}
```

3. Railway Platform Indicator

- **1) Statement:** A railway platform wants a flashing LED indicator to attract passengers' attention. Interface an LED with Arduino UNO and generate a pattern of three quick flashes followed by a 2-second pause, repeating continuously.
- **2) Hardware Details:** Arduino UNO, 1x LED, 1x 220Ω resistor, breadboard, wires.

- **3) Case Study Working:** The Arduino executes a sequence where the LED turns ON and OFF rapidly three times in succession, followed by a sustained 2-second OFF pause before repeating.
- **4) Schematic / Interfacing Diagram:** Arduino Digital Pin 7 \rightarrow 220Ω Resistor \rightarrow LED Anode. LED Cathode \rightarrow GND.
- **5) Pseudo Code:**

```
void setup() { pinMode(7, OUTPUT); }
void loop() {
  for(int i=0; i<3; i++) {
    digitalWrite(7, HIGH); delay(200);
    digitalWrite(7, LOW); delay(200);
  }
  delay(2000);
}
```

4. Classroom Light Control

- **1) Statement:** A classroom has a push button for controlling an LED-based light. When the push button is pressed, the LED should turn ON; when released, it should turn OFF.
- **2) Hardware Details:** Arduino UNO, 1x Push Button, 1x LED, 1x 220Ω resistor, 1x 10kΩ pull-down resistor, breadboard, wires.
- **3) Case Study Working:** The digital input pin continuously reads the state of the push button. When pressed, it detects a HIGH signal and immediately drives the LED output pin HIGH; when released, the pin goes LOW, turning the LED OFF.
- **4) Schematic / Interfacing Diagram:** Push button connected to 5V and Digital Pin 2 (with 10kΩ pull-down to GND). Arduino Digital Pin 8 \rightarrow 220Ω Resistor \rightarrow LED Anode \rightarrow GND.
- **5) Pseudo Code:**

```
void setup() { pinMode(2, INPUT); pinMode(8, OUTPUT); }
void loop() {
  if (digitalRead(2) == HIGH) { digitalWrite(8, HIGH); }
  else { digitalWrite(8, LOW); }
}
```

5. Security Door Indicator

- **1) Statement:** A security system uses a push button to represent a door sensor. When the switch is activated, the LED should turn ON as an indication that the door is open. When the switch is inactive, the LED should remain OFF.
- **2) Hardware Details:** Arduino UNO, 1x Push Button (door sensor), 1x LED, resistors, breadboard, wires.
- **3) Case Study Working:** The push button acts as a limit switch simulating a door. Pressing/activating the switch registers an open-door condition, lighting up the warning LED.
- **4) Schematic / Interfacing Diagram:** Button connected to Pin 4 and 5V. LED connected via resistor to Pin 10.

- **5) Pseudo Code:**

```
void setup() { pinMode(4, INPUT); pinMode(10, OUTPUT); }
void loop() {
  if (digitalRead(4) == HIGH) { digitalWrite(10, HIGH); }
  else { digitalWrite(10, LOW); }
}
```

6. Toggle-Based Lamp Control

- **1) Statement:** Design a lamp-control system in which pressing a push button changes the state of an LED. The first press turns the LED ON and the next press turns it OFF. Continue this operation for subsequent presses.
- **2) Hardware Details:** Arduino UNO, 1x Push Button, 1x LED, resistors, breadboard, wires.
- **3) Case Study Working:** Using state-change detection, the code tracks button presses and toggles an internal boolean state variable each time a new press transition is detected.
- **4) Schematic / Interfacing Diagram:** Button to Digital Pin 2. LED with series resistor to Digital Pin 8.
- **5) Pseudo Code:**

```
int lastState = LOW; boolean ledState = false;
void setup() { pinMode(2, INPUT); pinMode(8, OUTPUT); }
void loop() {
  int currentState = digitalRead(2);
  if (currentState == HIGH && lastState == LOW) {
    ledState = !ledState;
    digitalWrite(8, ledState);
    delay(50); // debounce
  }
  lastState = currentState;
}
```

7. Traffic Signal Status Indicator

- **1) Statement:** Use an RGB LED to represent three traffic conditions: Red \rightarrow STOP, Green \rightarrow GO, Blue \rightarrow WAIT/CAUTION. Program Arduino UNO to display each condition for a specified duration.
- **2) Hardware Details:** Arduino UNO, 1x Common Cathode/Anode RGB LED, 3x 220Ω resistors, breadboard, wires.
- **3) Case Study Working:** The microcontroller sequences through individual color channels of the RGB LED, illuminating Red, Green, and Blue for designated time periods sequentially.
- **4) Schematic / Interfacing Diagram:** RGB Red pin \rightarrow Pin 11, Green pin \rightarrow Pin 10, Blue pin \rightarrow Pin 9 (via current-limiting resistors). Common pin to GND.
- **5) Pseudo Code:**

```
void setup() { pinMode(11, OUTPUT); pinMode(10, OUTPUT);
pinMode(9, OUTPUT); }
void loop() {
  // Red (STOP)
  digitalWrite(11, HIGH); digitalWrite(10, LOW); digitalWrite(9,
LOW); delay(3000);
  // Green (GO)
  digitalWrite(11, LOW); digitalWrite(10, HIGH); digitalWrite(9,
LOW); delay(3000);
  // Blue (WAIT)
  digitalWrite(11, LOW); digitalWrite(10, LOW); digitalWrite(9,
HIGH); delay(1500);
}
```

8. Mood Lighting System

- **1) Statement:** Design a simple RGB mood-lighting system. The RGB LED should automatically cycle through red, green, blue, yellow, cyan and magenta, with each color displayed for 1 second.
- **2) Hardware Details:** Arduino UNO, 1x RGB LED, 3x 220Ω resistors, breadboard, wires.
- **3) Case Study Working:** Combinations of digital/PWM signals are sent to the Red, Green, and Blue pins simultaneously to generate primary and secondary colors (Yellow = Red + Green, Cyan = Green + Blue, Magenta = Red + Blue).
- **4) Schematic / Interfacing Diagram:** Red, Green, and Blue pins connected to Arduino PWM pins 9, 10, and 11 through resistors.
- **5) Pseudo Code:**

```
void setup() { pinMode(9, OUTPUT); pinMode(10, OUTPUT);
pinMode(11, OUTPUT); }
void loop() {
  // Red, Green, Blue, Yellow, Cyan, Magenta sequences
  // Example for Yellow (Red + Green):
  digitalWrite(9, HIGH); digitalWrite(10, HIGH); digitalWrite(11,
LOW); delay(1000);
  // ... add similar combinations for other colors
}
```

9. Variable Brightness Controller

- **1) Statement:** A potentiometer is connected between 5V and GND, with its wiper connected to an Arduino analog input. Read the analog voltage and control the brightness of an LED using PWM according to the potentiometer position.
- **2) Hardware Details:** Arduino UNO, 1x Potentiometer (10kΩ), 1x LED, 1x 220Ω resistor, breadboard, wires.
- **3) Case Study Working:** The analog pin reads a 0–1023 value from the potentiometer, which is mapped to a 0–255 PWM range and output to the LED pin to control its light intensity smoothly.

- **4) Schematic / Interfacing Diagram:** Potentiometer outer pins to 5V and GND, wiper to Analog Pin A0. LED anode connected to PWM Pin 9 via resistor.
- **5) Pseudo Code:**

```
void setup() { pinMode(9, OUTPUT); }
void loop() {
  int sensorValue = analogRead(A0);
  int pwmOutput = map(sensorValue, 0, 1023, 0, 255);
  analogWrite(9, pwmOutput);
}
```

10. Over-Voltage Warning System

- **1) Statement:** Design a voltage monitoring system using a potentiometer as a variable 0-5V input. If the measured voltage exceeds 2.5V, turn ON an LED; otherwise, keep it OFF.
- **2) Hardware Details:** Arduino UNO, 1x Potentiometer, 1x LED, 220Ω resistor, breadboard, wires.
- **3) Case Study Working:** The analog voltage read from the potentiometer is converted to a voltage scale. If it crosses the 2.5V threshold (ADC value > 512), the warning LED turns ON.
- **4) Schematic / Interfacing Diagram:** Potentiometer wiper to A0. LED connected to digital Pin 8.
- **5) Pseudo Code:**

```
void setup() { pinMode(8, OUTPUT); }
void loop() {
  int val = analogRead(A0);
  if (val > 512) { digitalWrite(8, HIGH); }
  else { digitalWrite(8, LOW); }
}
```

11. Three-Level Voltage Indicator

- **1) Statement:** Develop a system that monitors potentiometer voltage: Below 1.5V \rightarrow Green LED; 1.5–3.5V \rightarrow Yellow LED; Above 3.5V \rightarrow Red LED. Display the corresponding status using LEDs.
- **2) Hardware Details:** Arduino UNO, 1x Potentiometer, 3x LEDs (Red, Yellow, Green), resistors, breadboard, wires.
- **3) Case Study Working:** The input voltage from the potentiometer is measured and evaluated across three distinct threshold ranges to illuminate the appropriate status LED.
- **4) Schematic / Interfacing Diagram:** Potentiometer wiper to A0. Green LED on Pin 5, Yellow on Pin 6, Red on Pin 7.
- **5) Pseudo Code:**

```
void setup() { pinMode(5, OUTPUT); pinMode(6, OUTPUT); pinMode(7,
OUTPUT); }
void loop() {
  float voltage = analogRead(A0) * (5.0 / 1023.0);
```

```
  digitalWrite(5, voltage < 1.5);
  digitalWrite(6, voltage >= 1.5 && voltage <= 3.5);
  digitalWrite(7, voltage > 3.5);
}
```

12. Student Identification Display

- **1) Statement:** Interface a 16×2 LCD in 4-bit mode with Arduino UNO. Display the student's Name on Line 1 and Roll Number on Line 2.
- **2) Hardware Details:** Arduino UNO, 16x2 LCD Display, 10kΩ Potentiometer (for contrast), breadboard, wires.
- **3) Case Study Working:** The LiquidCrystal library initializes the LCD in 4-bit communication mode, printing custom string data to row 0 and row 1 respectively.
- **4) Schematic / Interfacing Diagram:** LCD RS, EN, D4, D5, D6, D7 connected to Arduino digital pins 12, 11, 5, 4, 3, 2.
- **5) Pseudo Code:**

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
void setup() {
  lcd.begin(16, 2);
  lcd.setCursor(0, 0); lcd.print("Ayush Kumar");
  lcd.setCursor(0, 1); lcd.print("Roll No: 101");
}
void loop() {}
```

13. Digital Welcome System

- **1) Statement:** Design a welcome display for a college laboratory. On power-up, the LCD should display: "WELCOME TO AIEC", "ARDUINO LAB". After 3 seconds, display the student's name and roll number.
- **2) Hardware Details:** Arduino UNO, 16x2 LCD display, potentiometer, breadboard, wires.
- **3) Case Study Working:** Initial welcome messages are shown on power-up. After a 3-second delay, the screen is cleared and replaced with student identification credentials.
- **4) Schematic / Interfacing Diagram:** Standard 4-bit LCD wiring to Arduino digital pins.
- **5) Pseudo Code:**

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
void setup() {
  lcd.begin(16, 2);
  lcd.print("WELCOME TO AIEC");
  lcd.setCursor(0, 1); lcd.print("ARDUINO LAB");
  delay(3000);
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print("Ayush Kumar");
  lcd.setCursor(0, 1); lcd.print("Roll: 101");
}
```

```
void loop() {}
```

14. Mini Information Display

- **1) Statement:** Interface a 16×2 LCD and display the following information sequentially: 1. College/Department name, 2. Student name, 3. Roll number, 4. Practical number. Each message should remain visible for 2 seconds.
- **2) Hardware Details:** Arduino UNO, 16x2 LCD, potentiometer, breadboard, wires.
- **3) Case Study Working:** The program loops through different text strings, clearing the screen or repositioning the cursor every 2 seconds to cycle through all required information fields.
- **4) Schematic / Interfacing Diagram:** Standard 4-bit LCD data and control pins connected to Arduino.
- **5) Pseudo Code:**

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
void setup() { lcd.begin(16, 2); }
void loop() {
  lcd.setCursor(0,0); lcd.print("Dept: CSE-AI"); delay(2000);
lcd.clear();
  lcd.setCursor(0,0); lcd.print("Name: Ayush"); delay(2000);
lcd.clear();
  lcd.setCursor(0,0); lcd.print("Roll: 101"); delay(2000);
lcd.clear();
  lcd.setCursor(0,0); lcd.print("Practical: 14"); delay(2000);
lcd.clear();
}
```

15. Smart Parking Alert

- **1) Statement:** Develop a parking assistance system using an ultrasonic sensor and buzzer. The buzzer should: Remain OFF when distance > 50 cm; Beep slowly between 20-50 cm; Beep rapidly below 20 cm. Simulate the system in Tinkercad.
- **2) Hardware Details:** Arduino UNO, HC-SR04 Ultrasonic Sensor, Buzzer, breadboard, wires.
- **3) Case Study Working:** The ultrasonic sensor measures object distance. Based on predefined distance brackets, the Arduino triggers buzzer beep intervals (slow, fast, or OFF).
- **4) Schematic / Interfacing Diagram:** Trig to Pin 7, Echo to Pin 6. Buzzer to Pin 8. VCC/GND connected appropriately.
- **5) Pseudo Code:**

```
void setup() { pinMode(7, OUTPUT); pinMode(6, INPUT); pinMode(8,
OUTPUT); }
void loop() {
  // Trigger pulse and read distance
  // Add logic for distance ranges (>50 off, 20-50 slow beep, <20
```

```
fast beep)
}
```

16. Obstacle Detection System

- **1) Statement:** An autonomous robot needs an obstacle warning system. Interface an HC-SR04 ultrasonic sensor and buzzer. When an obstacle comes within 15 cm, activate the buzzer; otherwise keep it OFF.
- **2) Hardware Details:** Arduino UNO, HC-SR04 Ultrasonic Sensor, Active Buzzer, breadboard, wires.
- **3) Case Study Working:** Constantly monitors obstacle distance. If distance drops below 15 cm, the buzzer turns ON to warn of the collision risk.
- **4) Schematic / Interfacing Diagram:** Trig \rightarrow Pin 9, Echo \rightarrow Pin 8, Buzzer \rightarrow Pin 10.
- **5) Pseudo Code:**

```
void setup() { pinMode(9, OUTPUT); pinMode(8, INPUT); pinMode(10,
OUTPUT); }
void loop() {
  // Measure distance via HC-SR04
  // if (distance < 15) digitalWrite(10, HIGH); else
digitalWrite(10, LOW);
}
```

17. Water Tank Level Warning

- **1) Statement:** Use an ultrasonic sensor to represent the water level of a tank. If the measured distance from the sensor is less than 10 cm, activate a buzzer indicating that the tank is nearly full.
- **2) Hardware Details:** Arduino UNO, HC-SR04 sensor, Buzzer, breadboard, wires.
- **3) Case Study Working:** The sensor measures the air gap above the water. When water rises and distance shrinks below 10 cm, the buzzer sounds an overflow warning.
- **4) Schematic / Interfacing Diagram:** Ultrasonic sensor pins connected to digital pins; Buzzer connected to digital output pin.
- **5) Pseudo Code:**

```
void setup() { /* Initialize pins */ }
void loop() {
  // Read distance; if distance < 10, buzzer ON, else OFF
}
```

18. Motion-Based Security Alarm

- **1) Statement:** Design a room-security system using a PIR sensor and buzzer. When motion is detected, the buzzer should turn ON for 5 seconds. When no motion is detected, the buzzer should remain OFF.
- **2) Hardware Details:** Arduino UNO, PIR Motion Sensor, Buzzer, breadboard, wires.

- **3) Case Study Working:** The PIR sensor detects infrared body heat movement. Upon detecting motion (HIGH output), the buzzer is activated for 5 seconds.
- **4) Schematic / Interfacing Diagram:** PIR output pin \rightarrow Arduino Digital Pin 2. Buzzer \rightarrow Pin 8.
- **5) Pseudo Code:**

```
void setup() { pinMode(2, INPUT); pinMode(8, OUTPUT); }
void loop() {
  if (digitalRead(2) == HIGH) {
    digitalWrite(8, HIGH); delay(5000);
  } else {
    digitalWrite(8, LOW);
  }
}
```

19. Smart Entry Alert

- **1) Statement:** A laboratory wants an alert whenever a person enters the room. Use a PIR sensor to detect movement and activate a buzzer. Also interface an LED that remains ON while motion is detected.
- **2) Hardware Details:** Arduino UNO, PIR Sensor, Buzzer, LED, resistor, wires.
- **3) Case Study Working:** Motion triggers both an audible buzzer and a visual LED indicator simultaneously while presence is detected in the entryway.
- **4) Schematic / Interfacing Diagram:** PIR to Pin 4, Buzzer to Pin 7, LED with resistor to Pin 8.
- **5) Pseudo Code:**

```
void setup() { pinMode(4, INPUT); pinMode(7, OUTPUT); pinMode(8,
OUTPUT); }
void loop() {
  if (digitalRead(4) == HIGH) {
    digitalWrite(7, HIGH); digitalWrite(8, HIGH);
  } else {
    digitalWrite(7, LOW); digitalWrite(8, LOW);
  }
}
```

20. Temperature Monitoring System

- **1) Statement:** Interface a TMP36 temperature sensor with Arduino UNO. Read the sensor output, calculate the temperature in °C, and display the temperature continuously on the Serial Terminal. If the temperature exceeds 35°C, display an additional warning message: "HIGH TEMPERATURE ALERT".
- **2) Hardware Details:** Arduino UNO, TMP36 Temperature Sensor, breadboard, USB cable, wires.
- **3) Case Study Working:** The analog voltage from the TMP36 is converted into temperature values in Celsius. These values print to the Serial Monitor, triggering an alert message if >35°C.

- **4) Schematic / Interfacing Diagram:** TMP36 VCC to 5V, GND to GND, Vout to Analog Pin A0.
- **5) Pseudo Code:**

```
void setup() { Serial.begin(9600); }
void loop() {
  int sensorVal = analogRead(A0);
  float voltage = sensorVal * (5.0 / 1023.0);
  float tempC = (voltage - 0.5) * 100.0;
  Serial.print("Temp: "); Serial.println(tempC);
  if (tempC > 35) { Serial.println("HIGH TEMPERATURE ALERT"); }
  delay(1000);
}
```

H/W Lab 1. Industrial Status Indicator

- **1) Statement:** An industrial control panel requires a visual status indicator. Interface an LED with Arduino UNO and program it to blink with a 1-second ON and 1-second OFF timing. Demonstrate the working circuit on hardware.
- **2) Hardware Details:** Physical Arduino UNO board, LED, 220Ω resistor, breadboard, USB cable.
- **3) Case Study Working:** The physical microcontroller outputs alternating HIGH and LOW voltage signals to flash the indicator LED every second.
- **4) Schematic / Interfacing Diagram:** Digital Pin 13 \rightarrow Resistor \rightarrow LED Anode \rightarrow GND.
- **5) Pseudo Code:**

```
void setup() { pinMode(13, OUTPUT); }
void loop() {
  digitalWrite(13, HIGH); delay(1000);
  digitalWrite(13, LOW); delay(1000);
}
```

H/W Lab 2. Student Information System

- **1) Statement:** Develop an Arduino UNO program that communicates with a computer through the USB Serial interface. Display student details (Name, Class/Division, Roll Number) in a properly formatted manner on the Serial Monitor.
- **2) Hardware Details:** Physical Arduino UNO board, USB cable, computer with Arduino IDE.
- **3) Case Study Working:** Upon startup, the Arduino transmits formatted student record strings over the serial communication channel to the connected PC.
- **4) Schematic / Interfacing Diagram:** Direct USB connection between Arduino UNO and computer.
- **5) Pseudo Code:**

```
void setup() {
  Serial.begin(9600);
  delay(1000);
```

```
  Serial.println("--- Student Info ---");
  Serial.println("Name: Ayush Kumar");
  Serial.println("Class: CSE-AI");
  Serial.println("Roll No: 101");
}
void loop() {}
```

H/W Lab 3. Laboratory Identification System

- **1) Statement:** Create a serial communication program that displays laboratory and student identification details every 2 seconds on the computer's Serial Terminal.
- **2) Hardware Details:** Physical Arduino UNO board, USB cable, computer.
- **3) Case Study Working:** The microcontroller periodically sends multi-line lab identification headers and student credentials to the serial terminal in a continuous loop.
- **4) Schematic / Interfacing Diagram:** USB cable connecting Arduino to PC.
- **5) Pseudo Code:**

```
void setup() { Serial.begin(9600); }
void loop() {
  Serial.println("ARDUINO UNO LAB");
  Serial.println("Name: Ayush");
  Serial.println("Class: CSE-AI");
  Serial.println("Roll No: 101\n");
  delay(2000);
}
```

H/W Lab 4. RGB LED - Three-Color Status Indicator

- **1) Statement:** Interface an RGB LED with Arduino UNO. Program it to sequentially display: RED \rightarrow GREEN \rightarrow BLUE, with each color remaining ON for 2 seconds.
- **2) Hardware Details:** Physical Arduino UNO, RGB LED, 3x 220Ω resistors, breadboard, wires.
- **3) Case Study Working:** By powering each color pin of the RGB LED individually while keeping others off, the hardware cycles through distinct red, green, and blue illuminated states.
- **4) Schematic / Interfacing Diagram:** Red pin \rightarrow Pin 11, Green pin \rightarrow Pin 10, Blue pin \rightarrow Pin 9 via resistors. Common pin to GND.
- **5) Pseudo Code:**

```
void setup() { pinMode(11, OUTPUT); pinMode(10, OUTPUT);
pinMode(9, OUTPUT); }
void loop() {
  digitalWrite(11, HIGH); digitalWrite(10, LOW);  digitalWrite(9,
LOW);  delay(2000);
  digitalWrite(11, LOW);  digitalWrite(10, HIGH); digitalWrite(9,
LOW);  delay(2000);
  digitalWrite(11, LOW);  digitalWrite(10, LOW);  digitalWrite(9,
```

```
HIGH); delay(2000);
}
```

H/W Lab 5. RGB-Based Device Status System

- **1) Statement:** Design a device-status indicator using an RGB LED: Red: System OFF / Error, Green: System Ready, Blue: System Processing. Program Arduino UNO to display the required colors sequentially on physical hardware.
- **2) Hardware Details:** Physical Arduino UNO, RGB LED, resistors, breadboard, wires.
- **3) Case Study Working:** Simulates machine status operational states by shifting RGB LED colors to signify Error (Red), Ready (Green), and Processing (active workflow in Blue).
- **4) Schematic / Interfacing Diagram:** RGB LED pins wired to Arduino digital PWM outputs with resistors.
- **5) Pseudo Code:**

```
void setup() { pinMode(9, OUTPUT); pinMode(10, OUTPUT);
pinMode(11, OUTPUT); }
void loop() {
  // Red (Error/OFF)
  digitalWrite(11, HIGH); digitalWrite(10, LOW); digitalWrite(9,
LOW); delay(2000);
  // Green (Ready)
  digitalWrite(11, LOW); digitalWrite(10, HIGH); digitalWrite(9,
LOW); delay(2000);
  // Blue (Processing)
  digitalWrite(11, LOW); digitalWrite(10, LOW); digitalWrite(9,
HIGH); delay(2000);
}
```